Simulated Smart-Home Control with Small Language Models on SBCs: Task Success, Latency, and Replication

Hudson Moreira Guimarães dos Santos*, Pablo Cesar Calcina Ccori*,
Laisa Caroline Costa De Biase*, Bruno Cavalcante de Souza Sanches*,
Roseli de Deus Lopes*, Marcelo Knörich Zuffo*
*University of São Paulo (USP), São Paulo, Brazil
Corresponding author: Hudson Moreira Guimarães dos Santos (hudson.moreira@usp.br)

***Abstract*—Small language models can broaden the phrasing accepted by a domestic controller, but their usefulness also depends on correct decisions and response cost. We compare a finite-rule baseline and quantized Qwen2.5 models of 0.5B and 1.5B parameters on Labrador single-board computers. Hardware and inference are real; the home and all actions are simulated. Forty reserved Portuguese scenarios cover clear instructions, paraphrases, routines, ambiguity, and invalid requests, with two repetitions per condition. Mean scenario success was 52.5% for rules, 37.5% for 0.5B, and 15.0% for 1.5B; request latency medians were 0.157, 36.848, and 94.663 s. All responses were structurally valid. The 0.5B configuration handled more paraphrases than rules, but both models struggled with rejection and clarification. Independent replicas on one, three, and six boards increased correct-response rates in fixed batches while retaining the same task-success counts. The study jointly evaluates decision quality and inference cost, distinguishing model proposals from executor interventions. Its conclusions concern the tested configurations and constructed workload.**

***Index Terms*—simulated smart home, small language models, SBC, task success, replication**

I. Introduction

A domestic controller must interpret a request, preserve unrelated device states, and decide when to ask for information or reject an unsupported operation. These requirements make natural-language control more demanding than producing a valid action description. On a resource-constrained device, correctness must also justify the time and memory required to obtain a response.

Two research directions inform this problem: evaluating domestic instructions, including invalid ones, and characterizing language models under edge resource constraints. Their intersection motivates a focused deployment question: how do rules and two small-model configurations differ in correct execution of simulated domestic tasks, inference cost, and service of independent requests through replicas?

We address this question using real ARM inference and a two-room lighting simulator. The contributions are a comparison against a finite-rule reference on the same Labrador boards; an evaluation that distinguishes valid output, correct proposals, and executor intervention; and instrumented closed batches served by independent replicas. Each board runs a complete model. The replicas process separate requests rather than collaborating on a decision or partitioning a model.

II. Related Work

HomeBench [1] evaluates English instructions in a virtual home, covering valid, invalid, and mixed requests involving one or multiple devices. Its execution-success metric requires all requested device operations to succeed; an additional F1 metric describes operation-level performance. It compares language models and adaptation strategies rather than a matched Labrador deployment. Our smaller Portuguese workload focuses on missing-room clarification and unrelated-state preservation, with separate records for proposed, blocked, and applied actions. We use an independently constructed dataset and do not compare our success percentages with HomeBench scores.

Lu et al. [2] characterize more than 60 small models using broad capability benchmarks and deployment-cost experiments. Capability evaluation uses a server, while cost evaluation includes smartphones and Jetson hardware, separating prefill and decoding and examining quantization. Our comparison instead measures task decisions and HTTP latency on the same CPU-only Labrador platform. It addresses a narrow control task rather than general capability rankings, connecting request cost to whether the resulting action is acceptable.

Qin et al. [3] examine how resource constraints affect personalized-model choices, including learning methods, data volume, compression, task difficulty, and available learning time. Their focus on customization differs from our fixed pretrained configurations: we perform inference without training or personalization. Together, these studies motivate joint evaluation of task requirements and deployment cost. Our addition is a traceable comparison of rules, small models, and independent replication for a Portuguese simulated-control workload. The comparisons are methodological; differences in tasks, models, language, and hardware preclude direct numerical rankings across studies.

III. Experimental Method

A. Simulator and execution policy

A Python coordinator creates a fresh state for each request, calls a resident HTTP service, validates its JSON, applies permitted changes, and independently evaluates the result. No domestic device is connected. The two rooms are identified by the technical values `sala` (living room) and `quarto` (bedroom). Lights take `on`/`off` values. The contract admits `set_light`, `set_lights`, `ask_clarification`, and `no_action`; a two-step plan is atomic, so one permission failure blocks the whole plan.

The executor sees only the request, visible state, and permissions, without access to evaluator answers or scenario categories. A conservative policy blocks lighting proposals if neither a known room name appears as a whole word in the request nor a valid location is supplied. Unknown rooms and unauthorized values are also blocked. This policy checks permission, not task correctness: a permitted proposal can still change the wrong light or misinterpret an invalid instruction. Proposals, intervention, and outcomes are recorded separately.

B. Hardware and configurations

Six Labrador boards provide four ARM64 CPU cores, Debian 12, and approximately 1.89 GiB of OS-visible RAM each. Models use `llama-server` [4] build 9584/`e25a32e98`, CPU execution, four compute and four batch threads, a 512-token context, and one slot per model service. Both model services stay resident on each board, with at most one inference active across them. Qwen2.5-0.5B-Instruct and Qwen2.5-1.5B-Instruct [5], [6] use Q5_K_M GGUF files whose hashes match the official Qwen files [7], [8]. Runtime and library hashes were checked across boards; full provenance is in the artifact.

Rules run as a resident stateless Python HTTP service on the same boards. The finite grammar recognizes fixed Portuguese verbs for turning lights on or off, explicit rooms, simple conjunctions, known-location commands, and politeness expressions at the ends of a request. Other forms return `no_action`. This is a limited engineering reference, not an optimal rule-based language system.

C. Workload and success criteria

Eight development cases preceded protocol freeze; development campaigns remain separate from the reserved evaluation. Scenarios, expected states and steps, prompts, schema, execution policy, model settings, repetitions, and measurement scripts were frozen before reserved outputs were available. The reserved set has eight scenarios in each of five categories: clear instructions, paraphrases, explicit routines, ambiguity, and invalid requests. Two repetitions give 80 requests per condition and 240 overall. Every condition runs across all hosts; repetitions of a scenario use different boards, with sequential requests on each board.

Model requests combine general English instructions with Portuguese user text. Temperature is zero, seeds are 801/802, output is limited to 128 tokens, and timeout is 180 s. Prompt caching is disabled. These repetitions assess operational stability, with limited decision diversity. After a timeout, the coordinator waits for both model services to be idle before another request.

Success requires normal completion, contract-valid JSON, the correct proposal, the expected final state, preservation of unrelated state, and no extra or prohibited steps. A single-light action and an equivalent one-step plan are accepted equally. Routines require exactly the two expected steps. Ambiguous requests require an informative question asking for the missing room and unchanged lights; invalid requests require `no_action` and unchanged lights. Thus, reaching a target state or blocking an incorrect proposal is insufficient on its own.

Clarification questions are checked using conservative full-question patterns; merely mentioning a room is insufficient. Other questions receive case-by-case review with identity and rationale retained. Four assisted judgments remain pending human validation and receive no success credit. The two repetitions are grouped within each scenario and scenarios receive equal weight. Request counts are not treated as independent scenarios for population inference.

D. Latency, memory, and replication

Coordinator monotonic time measures HTTP dispatch through response reading and decoding, including communication and inference but excluding model loading and SSH memory snapshots. Median and nearest-rank p95 describe individual requests. Runtime logs supply prompt and output tokens. RSS snapshots describe process memory before and after a request; VmHWM is a high-water mark since process startup. Neither is a per-request peak or exclusive physical model memory, because services remain resident and mappings may be shared.

Replication uses one, three, or six active boards, with one request per board per wave and two six-request batches per size and condition: 108 requests overall. Each batch contains one clear command, one paraphrase, two routines, one ambiguity, and one invalid request, each starting from a fresh state without conversation history. The batch window spans first dispatch to last HTTP response, including instrumented gaps between waves. We report responses/min and correct task responses/min separately. Simulation and grading follow receipt, so the latter is not measured time to state change. This closed workload evaluates batch service rather than saturation or maximum capacity.

IV. Results

A. Task success and inference cost

Table I summarizes 80 requests from 40 scenarios per condition. Latency is in seconds; the last two columns count individual light commands, including those inside plans, rather than replies. All 240 responses completed normally and satisfied the JSON contract. Success nevertheless differed substantially: rules completed 42/80 attempts, 0.5B completed 30/80, and 1.5B completed 12/80.

TABLE I
TASK SUCCESS, LATENCY, AND LIGHT COMMANDS.

Condition	Success	Median	p95	Undue	Blocked
Rules	42/80	0.157	0.643	0	0
Qwen 0.5B	30/80	36.848	45.292	36	16
Qwen 1.5B	12/80	94.663	159.117	76	22

TABLE II
TASK SUCCESS BY REQUEST CATEGORY.

Category	Rules	0.5B	1.5B
Clear instructions	16/16	8/16	0/16
Paraphrases	0/16	14/16	4/16
Routines	8/16	8/16	6/16
Ambiguity	2/16	0/16*	0/16
Invalid requests	16/16	0/16	2/16

The category comparison in Table II identifies where the approaches differed. Each cell covers eight scenarios with two repetitions, or 16 attempts; these are not 16 independent scenarios. Rules completed every clear and invalid request but none of the paraphrases. The 0.5B configuration completed 14/16 paraphrases, showing broader coverage of these formulations, yet rejected none of the invalid requests correctly. Neither model successfully handled ambiguity under the recorded criteria.

The asterisk marks four 0.5B clarification judgments awaiting human validation. They remain provisionally unsuccessful, and every credited success is automatic. Crediting all four would yield 34/80 (42.5%) for 0.5B, still below rules at 42/80 (52.5%); this sensitivity bound does not settle the judgments.

The 0.5B configuration also had lower request latency than 1.5B: medians were 36.848 and 94.663 s, versus 0.157 s for rules. Median post-request RSS was 571.6 and 1183.7 MiB for the models. Both used 124–138 prompt tokens; output ranges were 16–40 and 12–75 tokens, respectively. Recorded prompt-cache counts were zero. These measurements characterize the complete configurations rather than isolating the effect of parameter count.

B. Decision failures and representative cases

Table III groups existing evaluator checks without changing outcomes. Every entry gives failed/observed attempts. Step and target-state checks apply to 48 clear, paraphrase, and routine attempts; preservation applies to 64 attempts, excluding routines. Rows overlap and cannot be summed. Prohibition checks exist in all 80 attempts, although only 32 ambiguity/invalid attempts have nonempty prohibitions. Clarification and rejection rows evaluate proposals of those types; no clarification proposal is not evidence of clarification ability.

Rules preserved unrelated state but failed to act on valid requests outside their grammar. The models exhibited a different profile: incorrect step sets, prohibited proposals, and changes to unrelated lights. In particular, 1.5B failed step equality in 38/48 applicable attempts but the target-state check in 22/48, illustrating why the requested light alone cannot establish task completion. At command level, 0.5B proposed 36 undue changes, of which 16 were blocked and 20 simulated; 1.5B proposed 76, of which 22 were blocked and 54 simulated. These command counts differ from blocked-attempt counts because a reply may contain multiple commands and an entire plan is blocked atomically.

TABLE III
FAILED REQUIREMENTS AMONG OBSERVED ATTEMPTS.

Requirement	Rules	0.5B	1.5B
Wrong step set	24/48	18/48	38/48
Wrong target state	24/48	18/48	22/48
Preservation failed	0/64	8/64	14/64
Undue `no_action`	38/54	4/4	20/22
Proposal blocked	0/80	14/80	12/80
Prohibited step	0/80	26/80	24/80
Inadequate clarification	0/2	12/12*	–/0

Table IV presents four recorded cases. Requests and response text are English translations for presentation; the experiments used Portuguese. Technical JSON keys and values are retained, and action descriptions abbreviate the recorded JSON. Original utterances, complete outputs, states, and the mapping from case labels to attempt directories are preserved in the supplement. The inadequate-clarification count includes the four provisional judgments already identified.

Case 1 shows a valid paraphrase rejected by rules. Case 2 separates an incorrect room assumption from its blocked execution. Case 3 illustrates a question-field response that provisionally fails to ask for the missing information. Case 4 reaches the requested light state but changes the other room, failing preservation and exact-step criteria. These cases explain the evaluation distinctions without attributing unmeasured internal causes to the models.

C. Independent replicas

Table V and Fig. 1 describe batch service. Each table row pools 12 requests for success and latency; rates average two batch-level values. Latency is in seconds and rate is correct task responses/min over the measured HTTP window. For 0.5B, mean rates rose from 0.871 to 2.883 and 6.393/min with one, three, and six boards; 1.5B rose from 0.085 to 0.289 and 0.661/min. Each six-request batch retained three successes for 0.5B, one for 1.5B, and five for rules. The high rule rate with six replicas reflects a roughly 9–11 ms window and removal of between-wave instrumentation gaps, not measured sustained capacity. Batch windows and response rates are available separately in the artifact. Across the tested batches, replication increased concurrent service while success counts stayed unchanged. Together, the results distinguish three outcomes: all configurations met the output contract, task categories exposed different decision failures, and more replicas increased batch response rates without correcting those failures.

TABLE IV
REPRESENTATIVE DECISION FAILURES.

Case	Request and expected behavior	Observed response	Unmet requirement
1. **Unsupported paraphrase**	"Please leave the living-room light on." Turn on `sala`; preserve `quarto=on`.	Rules: `no_action`; reason translated as "Request outside the explicit rules." The requested light stays off.	Correct steps and target state; rejection is inappropriate for this valid request.
2. **Missing-room assumption**	"Turn on the light, please." No location is known; ask which room and preserve both lights.	0.5B: `set_light`, `room=sala`, `value=on`. Executor blocks the proposal; state is unchanged.	Correct action, permission, and absence of prohibited steps. Blocking prevents application but does not correct the proposal.
3. **Uninformative clarification**	"I want the light on." No location is known; ask which room and preserve both lights.	0.5B: `ask_clarification`, `missing_field=room`; `question` translated as "I want the light to turn on." No state change.	Informative question, provisionally: the assisted failure judgment awaits human validation.
4. **Unrequested state change**	"Turn on the light in the living room." Turn on `sala`; preserve `quarto=on`.	1.5B: `set_lights` with `(sala,on)` and `(quarto,off)`. Both commands are permitted and simulated.	Exact steps and preservation. The target light is correct, but the bedroom is also changed.

TABLE V
TASK RESPONSES WITH INDEPENDENT REPLICAS.

Condition	Boards	Success	Rate	Median	p95
Rules	1	10/12	42.753	0.006	0.211
Rules	3	10/12	235.196	0.006	0.008
Rules	6	10/12	30555.536	0.006	0.011
Qwen 0.5B	1	6/12	0.871	24.586	37.903
Qwen 0.5B	3	6/12	2.883	23.993	40.917
Qwen 0.5B	6	6/12	6.393	23.990	28.772
Qwen 1.5B	1	2/12	0.085	84.187	131.203
Qwen 1.5B	3	2/12	0.289	78.606	139.278
Qwen 1.5B	6	2/12	0.661	80.019	93.518

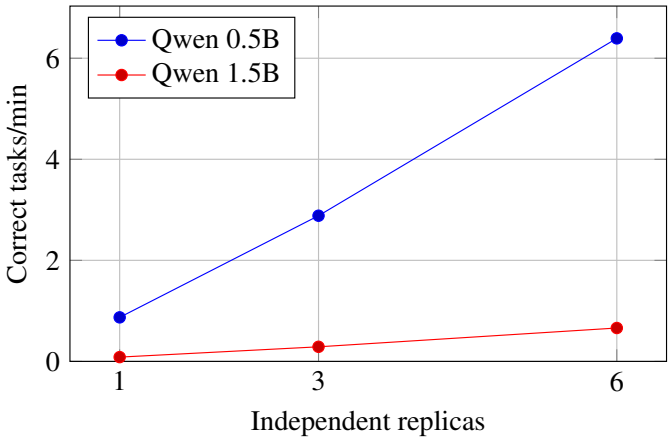


Fig. 1. Correct task response rates in closed batches.

V. DISCUSSION

A. Choosing an approach for the task

Rules offered the lowest response cost and the highest aggregate success in this workload, but their complete failure on paraphrases limits what that ranking means. The 0.5B configuration covered more of those requests while failing invalid-request rejection and several preservation checks. The practical choice therefore depends on which requests must be supported and which errors are acceptable, alongside latency and memory budgets. The current evidence does not establish a reliable combined controller; such a design would require independent validation.

The distinction between syntax and execution complements HomeBench's emphasis on complete instruction execution [1]. Our missing-room and preservation cases further show how a blocked proposal or a correct target light can mask an unsuccessful decision. Their value is diagnostic, not evidence that our smaller workload is more comprehensive than HomeBench.

The cost comparison extends the deployment perspective of Lu et al. [2] to this Labrador task: measuring a model's footprint or generating valid JSON is insufficient to choose it for control. The slower 1.5B configuration also completed fewer tasks here. That observation concerns the tested prompt, quantization, and models; it does not identify a size-dependent cause. Qin et al. [3] likewise frame selection around task and resource constraints, but their customization setting cannot explain our model ranking. Testing an adapted model would require new development data and a separately frozen protocol.

Replication addresses a separate deployment decision. Parallel copies increased correct-response rates for independent requests, with similar individual medians across replica counts and unchanged success counts in each batch. They did not im-

prove interpretation of a request. Evaluating sustained service would require an arrival process, queue measurements, and longer runs beyond these instrumented batches.

B. Validity and scope

The 40 authored utterance/state combinations cover two lights in Portuguese, not independent homes or a population of users. Several express the same underlying action. Requested lights start in the opposite state, leaving idempotent requests and other state distributions untested; routine success alone cannot distinguish language understanding from state inversion. The finite grammar is a limited baseline, and one prompt, schema, and quantization per model do not isolate architecture or size effects.

Host interleaving reduces confounding without eliminating thermal, memory, or network variation. Resident services and shared mappings limit memory interpretation; page caches were not cleared, and energy and per-request peaks were not measured. Few closed batches, including instrumentation costs, limit service-rate generalization. Four assisted judgments remain provisional. Simulation provides no evidence of real-home safety or usability. These limits define the scope of the configuration comparison rather than changing its observed outcomes.

VI. Conclusion

For the tested simulated domestic tasks, rules achieved the highest aggregate success at the lowest response cost, while Qwen 0.5B handled more paraphrases than the finite grammar and outperformed the tested 1.5B configuration in both success and latency. Valid structured output did not guarantee correct decisions: rejection, clarification, and preservation required separate evaluation. Independent replicas increased correct-response rates in closed batches while leaving task-success counts unchanged.

Selecting a controller therefore requires matching supported requests and decision criteria to the available inference budget. The principal limit is the small authored two-room workload. A concrete next step is a new, independently reviewed evaluation with user-authored phrasing, idempotent commands, and varied initial states, retaining the distinction between proposed actions, executor intervention, and task success.

Evidence Availability and AI-use Disclosure

The frozen protocol, raw responses, all 240 quality and 108 replication attempts, failures, review ledger, and offline reconstruction scripts are available at an immutable snapshot [9]. The complementary failure analysis preserves the original examples and their source paths. The label mapping and translation notes accompany this editorial revision in the repository. Offline reconstruction checks recorded state, permissions, and outcomes without new inference; independent hardware replication additionally requires the recorded runtime and model files.

No external funding was received. OpenAI Codex assisted code, analysis, figures, evidence and literature checks, and manuscript drafting and revision. Its four provisional clarification judgments are not human review. Authors remain responsible for verification and the final text under IEEE AI-use guidance [10].

References

[1] S. Li, Y. Guo, J. Yao, Z. Liu, and H. Wang, "HomeBench: Evaluating LLMs in smart homes with valid and invalid instructions across single and multiple devices," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2025, pp. 12 230–12 250. [Online]. Available: https://aclanthology.org/2025.acl-long.597/

[2] Z. Lu, X. Li, D. Cai, R. Yi, F. Liu, W. Liu, J. Luan, X. Zhang, N. D. Lane, and M. Xu, "Demystifying small language models for edge deployment," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2025, pp. 14 747–14 764. [Online]. Available: https://aclanthology.org/2025.acl-long.718/

[3] R. Qin, D. Liu, C. Xu, Z. Yan, Z. Tan, Z. Jia, A. Nassereldine, J. Li, M. Jiang, A. Abbasi, J. Xiong, and Y. Shi, "Empirical guidelines for deploying LLMs onto resource-constrained edge devices," *ACM Transactions on Design Automation of Electronic Systems*, vol. 30, no. 5, 2025, art. 70. [Online]. Available: https://doi.org/10.1145/3736721

[4] ggml-org, "llama.cpp server documentation," accessed: 2026-10-20; deployed source revision e25a32e98c15a1118a976685af0f4d2c1f380079. [Online]. Available: https://github.com/ggml-org/llama.cpp/blob/e25a32e98c15a1118a976685af0f4d2c1f380079/tools/server/README.md

[5] Qwen, "Qwen2.5-0.5B-Instruct: Official model card," accessed: 2026-10-05. [Online]. Available: https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct

[6] ——, "Qwen2.5-1.5B-Instruct: Official model card," accessed: 2026-10-05. [Online]. Available: https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct

[7] ——, "Qwen2.5-0.5B-Instruct Q5_K_M: Official GGUF file and SHA-256," accessed: 2026-10-06. [Online]. Available: https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct-GGUF/blob/main/qwen2.5-0.5b-instruct-q5_k_m.gguf

[8] ——, "Qwen2.5-1.5B-Instruct Q5_K_M: Official GGUF file and SHA-256," accessed: 2026-10-06. [Online]. Available: https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct-GGUF/blob/main/qwen2.5-1.5b-instruct-q5_k_m.gguf

[9] Labrador Distributed LLM Benchmarks contributors, "Simulated smart-home control: Frozen protocol, raw attempts, and reconstruction scripts," GitHub repository (snapshot a834b82), 2026, evidence snapshot a834b82, accessed 2026-10-20.

[10] IEEE, "Guidelines for artificial intelligence (AI)-generated text," 2024, accessed: 2026-10-20. [Online]. Available: https://open.ieee.org/author-guidelines-for-artificial-intelligence-ai-generated-text/